

LARGE LANGUAGE MODELS

Training Pipeline, Architecture, and Emergent Behavior

Introduction

Large Language Models (LLMs) such as GPT, BERT, and T5 have become the foundation of modern natural language processing (NLP). They are trained on massive text corpora and learn statistical patterns of language that allow them to generate, understand, and transform text with human-like fluency. This report documents five core tasks completed during the internship: the end-to-end LLM training pipeline, the mechanics of next-token prediction, a comparison of major transformer architectures, the reasoning behind autoregressive generation, and the emergent capabilities that appear as models scale. Each section combines conceptual explanation with concrete examples, diagrams, and comparison tables to demonstrate a practical understanding of how these models are built and why they behave the way they do.

Task 1 — Complete LLM Training Pipeline

Training a large language model is a multi-stage process that transforms raw, unstructured text into a system capable of fluent, context-aware generation. Each stage below builds on the output of the previous one. The end-to-end flow is summarized in the diagram and then explained stage by stage.

Raw Data	Cleaning	Tokenization	Embeddings	Positional Encoding	Pre-training	SFT	RLHF	Production Model
----------	----------	--------------	------------	---------------------	--------------	-----	------	------------------

1. Raw Data Collection

Training begins with gathering very large volumes of publicly available text: books, research papers, websites, Wikipedia articles, news archives, code repositories, and educational material. Modern frontier models are trained on corpora ranging from hundreds of billions to several trillion tokens. The goal is broad coverage — diverse writing styles, domains, and factual knowledge — so the model develops general-purpose language understanding rather than narrow, task-specific skills.

2. Data Cleaning

Raw web-scale data is noisy: it contains duplicate pages, boilerplate navigation text, advertisements, HTML/markup artifacts, corrupted files, spam, and low-quality or toxic content. Cleaning pipelines apply deduplication, language filtering, quality classifiers, and boilerplate removal so the model learns from reliable, information-dense text rather than memorizing noise. Data quality has a direct effect on downstream model quality — this is why cleaning is treated as its own dedicated stage rather than a minor preprocessing step.

3. Tokenization

Neural networks operate on numbers, not raw text, so a tokenizer first breaks text into smaller units called tokens (sub-words, whole words, or characters) and maps each one to a numerical ID from a fixed vocabulary. Most modern LLMs use sub-word algorithms such as Byte-Pair Encoding (BPE) or Word Piece, which balance vocabulary size against the ability to represent rare or unseen words by splitting them into familiar fragments.

4. Embeddings

Each token ID is converted into a dense numerical vector (typically several hundred to a few thousand dimensions) that captures semantic meaning. These embedding vectors are learned during training such that words used in similar contexts end up close together in vector space — for example, "king" and "queen" or "happy" and "joyful" will have vectors that are near one another, allowing the model to generalize across related concepts.

5. Positional Encoding

The Transformer architecture processes all tokens of a sequence in parallel rather than one at a time, which means it has no inherent sense of word order. Positional encodings — fixed sinusoidal patterns or learned positional embeddings — are added to the token embeddings so the model can distinguish "the dog bit the man" from "the man bit the dog".

6. Pre-training (Self-Supervised Learning)

During pre-training, the model repeatedly performs next-token prediction across billions or trillions of tokens. For each position in a sequence, it outputs a probability distribution over the entire vocabulary; the difference between this prediction and the actual next token is measured using cross-entropy loss, and the model's parameters (often tens to hundreds of billions of them) are updated using gradient descent and backpropagation. This stage requires no manual labelling — the "label" for each position is simply the next word in the existing text, which is why it is called self-supervised learning.

7. Supervised Fine-Tuning (SFT)

After pre-training, the base model is fine-tuned on curated prompt–response pairs written or reviewed by humans. This teaches the model the conversational and instructional format expected of an assistant: answering questions directly, summarizing text, drafting emails, explaining concepts, and following multi-step instructions, rather than simply continuing text in a raw, unstructured way.

8. Reinforcement Learning from Human Feedback (RLHF)

Human evaluators rank multiple candidate responses to the same prompt from best to worst. These preference rankings are used to train a separate reward model, which then guides further optimization of the language model (commonly via algorithms such as PPO) so that it produces responses that are more helpful, honest, and harmless. RLHF is what shapes tone, safety behavior, and alignment with human expectations, on top of the raw language ability learned during pre-training.

9. Evaluation, Safety Testing, and Deployment

Before release, models are evaluated on benchmark suites (reasoning, factuality, coding, safety) and red-teamed to probe for harmful or unreliable behavior. Once approved, the model is deployed for inference: a user prompt is tokenized, passed through the Transformer, and the model generates output tokens one at a time until a stopping condition (such as an end-of-sequence token) is reached.

Task 2 — Next-Token Prediction

Next-token prediction is the core mechanism that drives text generation in autoregressive LLMs. At every step, the model takes the tokens generated so far as context and outputs a probability distribution over its entire vocabulary (via a softmax layer). The token with the highest probability — or one sampled according to a decoding strategy such as temperature sampling or top-k/top-p sampling — is selected, appended to the sequence, and fed back in as part of the context for the next prediction.

Worked example using the sentence "I love artificial intelligence.":

Step	Input Context	Predicted Next Token
1	I	love
2	I love	artificial

Step	Input Context	Predicted Next Token
3	I love artificial	intelligence
4	I love artificial intelligence	.

This confirms that the model never generates a full sentence in one shot. Each prediction is conditioned on everything generated so far, and the process repeats — one token at a time — until a complete, coherent response is produced. This is mathematically expressed as decomposing the probability of a whole sequence into a product of conditional probabilities, one per token (detailed further in Task 5).

Task 3 — GPT vs BERT vs T5

Although GPT, BERT, and T5 are all built on the Transformer architecture, they differ in which half of the Transformer they use (encoder, decoder, or both) and how they are pre-trained — and this determines what each model is best suited for.

GPT (Decoder-only)

- Uses only the Transformer decoder with causal (left-to-right) self-attention.
- Pre-trained purely on next-token prediction, so it is naturally a text generator.
- Best suited for chatbots, coding assistants, creative writing, and open-ended content generation.

BERT (Encoder-only)

- Uses only the Transformer encoder, which attends to both left and right context simultaneously (bidirectional).
- Pre-trained using Masked Language Modelling (predicting randomly hidden words) and Next Sentence Prediction.
- Best suited for tasks that require deep understanding rather than generation: sentiment analysis, text classification, named entity recognition, and semantic search.

T5 (Encoder–Decoder)

- Uses the full Transformer: an encoder to read the input and a decoder to generate the output.
- Frames every NLP problem — translation, summarization, classification, question answering — as a unified "text-to-text" task.
- Best suited for translation, summarization, question answering, and paraphrasing, where an input sequence must be transformed into a related but different output sequence.

Side-by-Side Comparison

Aspect	GPT (Decoder-only)	BERT (Encoder-only)	T5 (Encoder–Decoder)
Architecture	Decoder only	Encoder only	Encoder + Decoder
Attention	Causal (left-to-right)	Bidirectional	Bidirectional (encoder) + causal (decoder)
Pre-training objective	Next-token prediction	Masked language modelling	Text-to-text / span corruption

Aspect	GPT (Decoder-only)	BERT (Encoder-only)	T5 (Encoder–Decoder)
Primary strength	Generating text	Understanding text	Transforming text
Typical use cases	Chatbots, code generation	Classification, search, NER	Translation, summarization, Q&A

Task 5 — Why LLMs Are Autoregressive

An autoregressive language model generates text by predicting one token at a time, where each new token is conditioned on all previously generated tokens. Formally, the probability of an entire sequence of tokens is decomposed using the chain rule of probability into a product of conditional probabilities

This is why the model never generates an entire sentence in a single forward pass — it would need to know the whole answer before predicting the very first word. Instead, generation proceeds sequentially

Each predicted token is appended to the existing sequence and becomes additional context for the next prediction step. This autoregressive loop is what allows a model with a fixed context window to produce coherent paragraphs, essays, or code of arbitrary length, since the sequence simply keeps growing one token at a time. It is worth noting that this contrasts with non-autoregressive approaches used in some specialized generation models, which attempt to produce multiple tokens in parallel to trade some quality for speed — a design choice that is far less common in general-purpose LLMs because it sacrifices the strong conditioning that gives autoregressive models their coherence.

Task 6 — Emergent Capabilities

Emergent capabilities are abilities that appear, often quite abruptly, as models are scaled up in parameter count, training data, and compute — despite not being explicitly programmed or directly trained for those exact tasks. This behavior is closely tied to "scaling laws," which describe how model performance improves predictably with scale, and to the observation that certain capabilities only cross a usable threshold once a model passes a certain size.

1. Few-shot learning — the model can learn a new task from just a handful of examples shown directly in the prompt, without any additional training.
2. In-context learning — the model adapts its behavior to a new task purely from the prompt itself, with no updates to its underlying weights.
3. Arithmetic and logical reasoning — larger models can perform multi-step mathematical operations and logical deductions through learned patterns, and techniques such as chain-of-thought prompting further improve this ability.
4. Code generation — the model can write, explain, debug, and translate programs across multiple programming languages, despite code being only one portion of its training data.
5. Multilingual ability — the model can understand and generate text across many languages, including some it saw relatively little of during training, by transferring patterns learned from higher-resource languages.

These behaviors consistently become stronger and more reliable as model size, data quality, and training compute increase, which is why scaling has been such a central strategy in modern LLM development — though

it also raises open research questions about predictability, evaluation, and the safety implications of capabilities that were not explicitly designed for.

Conclusion

The complete LLM pipeline begins with collecting and cleaning large-scale text datasets, followed by tokenization, embeddings, and positional encoding to prepare the data for the Transformer architecture. Large-scale self-supervised pre-training teaches the model general language patterns through next-token prediction, while Supervised Fine-Tuning and RLHF align the model with human instructions, preferences, and safety expectations before deployment. Architecturally, the choice between decoder-only (GPT), encoder-only (BERT), and encoder-decoder (T5) designs determines whether a model is optimized for generation, understanding, or transformation of text. Because generation is fundamentally autoregressive — each token conditioned on everything before it — models are able to produce long, coherent outputs one prediction at a time. Finally, as these models are scaled up, they exhibit emergent capabilities such as few-shot learning, reasoning, code generation, and multilingual understanding that were not explicitly engineered, underscoring both the promise and the ongoing research challenges of large language models.